PROYECTO INTEGRADO

Comparador web de precios de supermercados con integración de IA

Álvaro G.
IES FRANCISCO AYALA

ABSTRACT

El proyecto consiste en una aplicación web para comparar precios de productos entre distintos supermercados de España, permitiendo al usuario saber donde le sale más barata la compra. El sistema utiliza inteligencia artificial para normalizar los datos, identificando equivalencias entre productos, aunque cada cadena los nombre de una forma diferente.

Dentro de las funciones que incluye la web, los usuarios pueden buscar productos específicos, comparar precios por unidad y añadir artículos a una lista de la compra para calcular en qué supermercado es más económica su cesta completa. Además, permite configurar alertas para recibir avisos por Telegram cuando un producto baje de precio.

Inglés:

The project consists of web application to compare product pricess across different supermarkets in Spain, allowing users to find the cheapest place to shop. The system uses artifical intelligence to normalize data, identifying equivalent products even if each chain names them differently.

Among the features included in the website, users can search for specific products, compare unit prices, and add ítems to a shopping list to calculate which supermarket offers the most economical total for their basket. Additionally, it allows setting up alerts to receive notifications via Telegram when a product price drop.

ÍNDICE

Contenido

1. Justificación 3
   1.1 Características Generales 3
   1.2 Restricciones generales 3
   1.3 Aspectos a cubrir y no cubiertos 4
2. Justificación de la tecnología empleada 4
3. Requerimientos Hardware y Software 5
4. Análisis y diseño 6
   4.1 Diagrama Casos de uso y Estructura del sistema 6
      4.1.1 Diagrama Casos de uso 6
      4.1.2 Estructura del código 7
   4.2 Base de Datos Relacional 7
5. Implementación 8
   5.1 Automatización con n8n 8
      5.1.1 Normalización de productos con IA (OpenAI) 8
      5.1.2 Flujo de Alertas de Telegram 9
      5.1.3 Flujo de Consultas vía Bot de Telegram 9
      5.1.4 Flujo de procesamiento 10
      5.1.5 Flujo de inserción de productos 11
6. Evaluación y prueba 11
   6.1 Verificación de funciones 11
   6.2 Limitaciones detectadas (Errores de IA) 12
7. Capturas de pantalla 12
   7.1 Inicio 12
   7.2 Resultados 13
   7.3 Lista compra 13
   7.4 Perfil 14
   7.5 Inicio de sesión 15
   7.6 Registro 15
   7.7 Interfaz del Bot de Telegram 16
8. Mapa de navegación 17
9. Conclusiones y Mejoras posibles 17
   9.1 Mejoras posibles y aportaciones 17

1. Justificación

1.1 Características Generales

La aplicación web ComparaYA nace como una herramienta para ayudar al ahorro familiar frente a la subida general de precios. Su objetivo es permitir al usuario comparar cualquier tipo de artículo entre los principales supermercados de España. Sus características principales son:

- **Buscador con IA**: Utiliza Inteligencia Artificial para normalizar los nombres de los productos y encontrar equivalencias entre distintos supermercados.

- **Comparativa de cesta**: Permite añadir varios productos a una lista y calcular automáticamente en qué supermercado sale más barata la compra completa.

- **Alertas personalizadas**: Los usuarios registrados pueden configurar alertas para recibir un aviso directamente en su móvil a través de un bot de Telegram cuando un producto baja de precio.

1.2 Restricciones generales

Al ser un proyecto en fase de prototipo, existen las siguientes limitaciones:

- **Número de supermercados**: Actualmente el sistema trabaja con una selección de dos cadenas principales para validar que la arquitectura y la IA funcionan correctamente en este caso son DÍA y Mercadona.

- **Actualización de datos**: Debido a la complejidad de las fuentes de origen, la actualización de precios no se realiza de forma diaria automática, quedando como un proceso manual en esta fase.

- **Catálogo de productos**: No se ha extraído el 100% de los productos de los supermercados, centrándose en una base de datos representativa para las pruebas. Aunque hay una gran cantidad de productos.

1.3 Aspectos a cubrir y no cubiertos

Aspectos a cubrir:

- Comparación de precios de productos entre los supermercados integrados.
- Uso de IA para la limpieza de datos y agrupación de productos idénticos.
- Gestión de usuarios, listas de la compra y sistema de alertas vía Telegram.

Aspectos no cubiertos:

- Actualización automática de precios cada día mediante tareas programadas
- Alertas por correo electrónico: el sistema de alertas se centra exclusivamente en Telegram por ahora
- Gestión de alertas tras notificación: En esta fase, las alertas permanecen activas tras ser enviadas. La eliminación automática una vez notificado el usuario queda para la siguiente versión

2. Justificación de la tecnología empleada

Las tecnologías utilizadas en el proyecto son React, Node.js, MySQL, Python y n8n. Este conjunto de herramientas se usa para crear una web funcional y capaz de gestionar datos de diferentes supermercados.

Frontend:

- **React**: Framework utilizado para crear la interfaz de usuario. Permite que la web sea dinámica y fácil de usar para el cliente.
- **Tailwind CSS**: Framework de diseño utilizado junto con React para el estilo visual de la web. Me ha permitido crear una interfaz moderna, responsive (que se adapta a móviles) y limpia de forma mucho más rápida y organizada.

Backend:

- **Node.js con Express**: Utilizado para gestionar las peticiones de la web y conectar con la base de datos.

Automatización y Datos:

Python usando playwright (Scrapping): Para sacar los productos de los dos supermercados, he utilizado y adaptado un script de Python que utilizo en n8n que he encontrado en GitHub (https://github.com/joseluam97/Supermarket-Price-Scraper) Esto me ha permitido obtener una base de datos real para trabajar.

Integración de IA (n8n + OpenAI): La normalización no es un proceso manual. He configurado un flujo en n8n que incluye un nodo de OpenAI. Este nodo recibe los datos

en bruto del scraping, les aplica un prompt específico y devuelve el producto clasificado y normalizado. Esto permite procesar miles de productos de forma automática y coherente.

- **Infraestructura (Hostinger/Linux)**: Para que el sistema funcione 24/7 sin depender de mi ordenador personal, el backend y los flujos de n8n están alojados en un servidor VPS con Linux en Hostinger.

Base de datos:

- **MySQL**: Utilizada para el almacenamiento de toda la información de productos, precios y usuarios registrados.

Seguridad y Gestión:

- **Firebase**: Utilizado para la gestión de usuarios, garantizando un control de acceso seguro y eficiente.

Comunicación:

- **Bot de Telegram**: Implementado para que los usuarios reciban notificaciones en tiempo real cuando un producto baja de precio.

3. Requerimientos Hardware y Software

Requerimientos de Software

- **Node.js**: Necesario para ejecutar el servidor backend y gestionar los paquetes del frontend.

- **MySQL**: Sistema de gestión de base de datos para almacenar productos, precios y las listas de la compra.

- **n8n**: Plataforma de automatización donde se ejecutan los scripts de Python para el scraping y la normalización con IA.

- **Firebase**: Plataforma necesaria para la gestión y autenticación de los usuarios del sistema.

- **Navegador Web**: Cualquier navegador moderno para acceder a la aplicación desarrollada en React.

Requerimientos de Hardware

- **Servidor**: Capacidad suficiente para procesar la base de datos y el servidor Node.js al mismo tiempo.

- **Conexión a Internet**: Imprescindible para conectar con Firebase, n8n y para que el scraping de los supermercados funcione

4. Análisis y diseño

4.1 Diagrama Casos de uso y Estructura del sistema

4.1.1 Diagrama Casos de uso

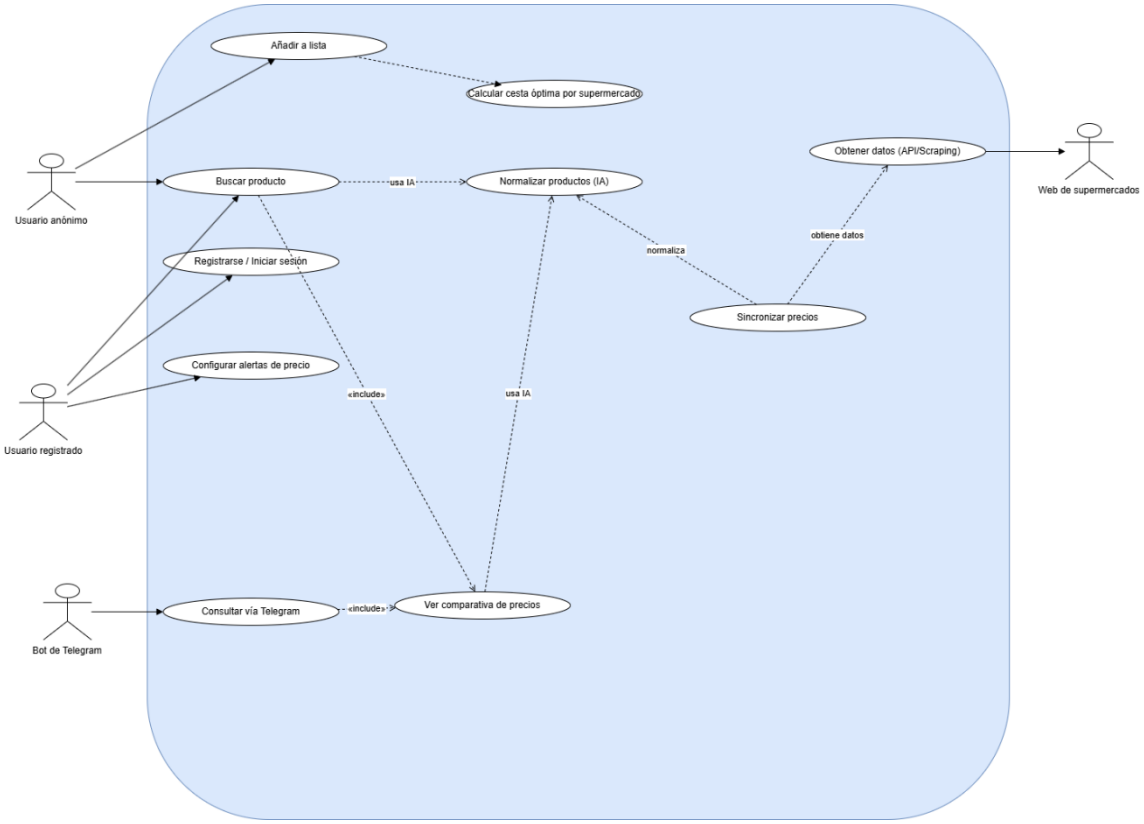
En el sistema intervienen los siguientes actores principales:

- **Usuario anónimo**: Puede buscar productos y ver las comparaciones de precios.

- **Usuario registrado**: Además de lo anterior, puede guardar los productos en su lista de la compra y configurar las alertas de precio.

- **Sistema (n8n)**: Es la parte interna que se encarga de obtener los precios de las webs de los supermercados y normalizar los datos usando IA.

- **Bot de Telegram**: Se utiliza como canal directo para enviar las alertas al móvil del usuario.



4.1.2 Estructura del código

Como el proyecto está desarrollado con React y Node.js, no se ha utilizado un esquema de clases tradicional. En su lugar, el código se ha organizado siguiendo una adaptación del patrón Modelo-Vista-Controlador (MVC). Esto permite que el código no esté todo mezclado, separando la lógica de los datos y de la interfaz visual:

Backend (Modelos y Controladores): El servidor está organizado en carpetas siguiendo este patrón. Tenemos la carpeta routes que recibe las peticiones de la web, los controllers donde está la lógica principal (buscar, comparar, gestionar listas), y los models, que son los archivos que se conectan directamente con la base de datos MySQL para sacar la información.

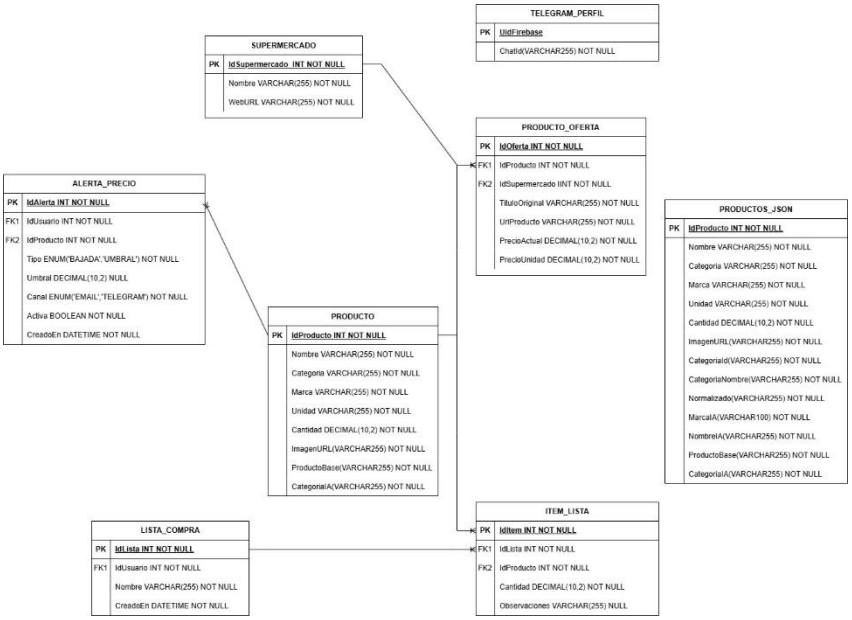
Frontend (Vistas): La interfaz visual, hecha en React, actúa como la "Vista" para el usuario. Está dividida en pages (las pantallas completas como Inicio, Perfil o Resultados) y components (elementos que se repiten, como las tarjetas de productos, la barra de búsqueda o los modales).

4.2 Base de Datos Relacional

El sistema utiliza una base de datos relacional para organizar toda la información de las tiendas y los usuarios.

La base de datos se centra en la tabla PRODUCTO, que representa un artículo de forma genérica, por ejemplo, "Manzana". Como ese mismo producto puede estar en varios sitios, utilizamos la tabla PRODUCTO_OFERTA para relacionarlo con un supermercado concreto y guardar a qué precio lo tiene. También tenemos la tabla PRODUCTOS_JSON que sirve para guardar los datos recién extraídos de internet antes de que la IA los limpie.

Por otro lado, para los usuarios registrados tenemos la tabla LISTA_COMPRA que guarda su cesta actual, y la tabla ITEM_LISTA con los productos que tiene dentro. Por último, los avisos se guardan en ALERTA_PRECIO y cruzamos los datos en TELEGRAM_PERFIL para saber a qué chat enviar el mensaje.



5. Implementación

En este apartado se explica cómo se ha construido la aplicación y cómo se conectan las distintas partes para que todo funcione.

- **Frontend**: He utilizado React para crear la interfaz de la web. Es lo que permite que el buscador sea rápido y que el usuario vea las comparaciones de precios al momento.

- **Backend**: He usado Node.js con Express para gestionar las peticiones de la web y conectar con la base de datos.

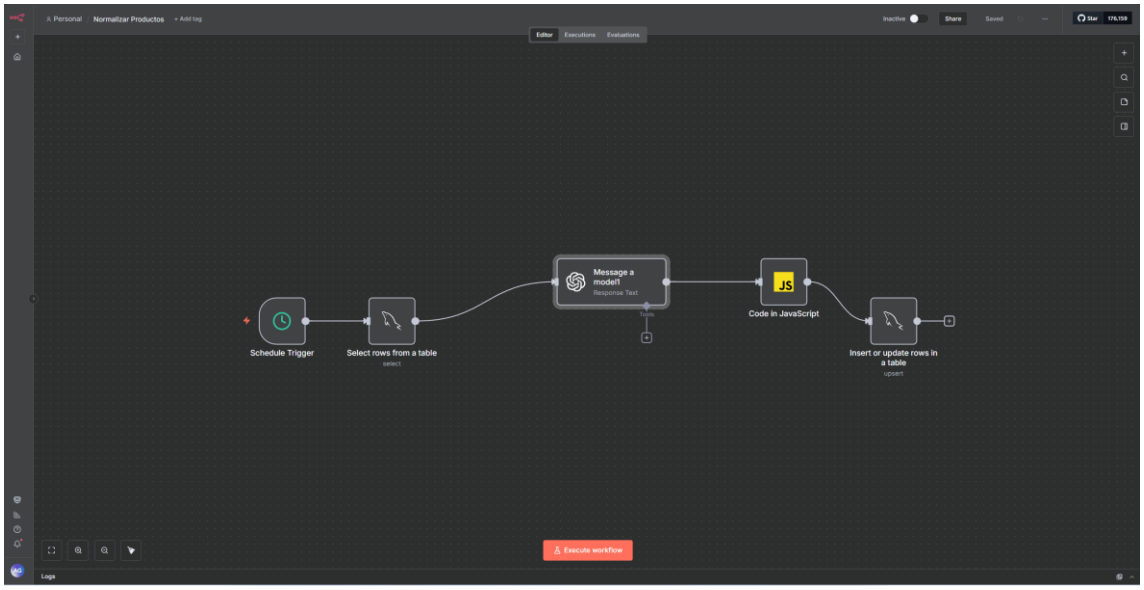
- **Gestión de Usuarios**: Para que el registro sea seguro y fácil, he integrado Firebase para el control de cuentas de los usuarios.

- **Flujo de Datos y Automatización**: Esta es la parte clave donde uso n8n. He configurado flujos que procesan los productos sacados con el script de Python que he encontrado en GitHub, aplican la IA para la normalización y mandan los avisos al bot de Telegram.

- **Base de Datos**: Uso MySQL para guardar toda la información de los productos de los dos supermercados y los precios que se van sacando.

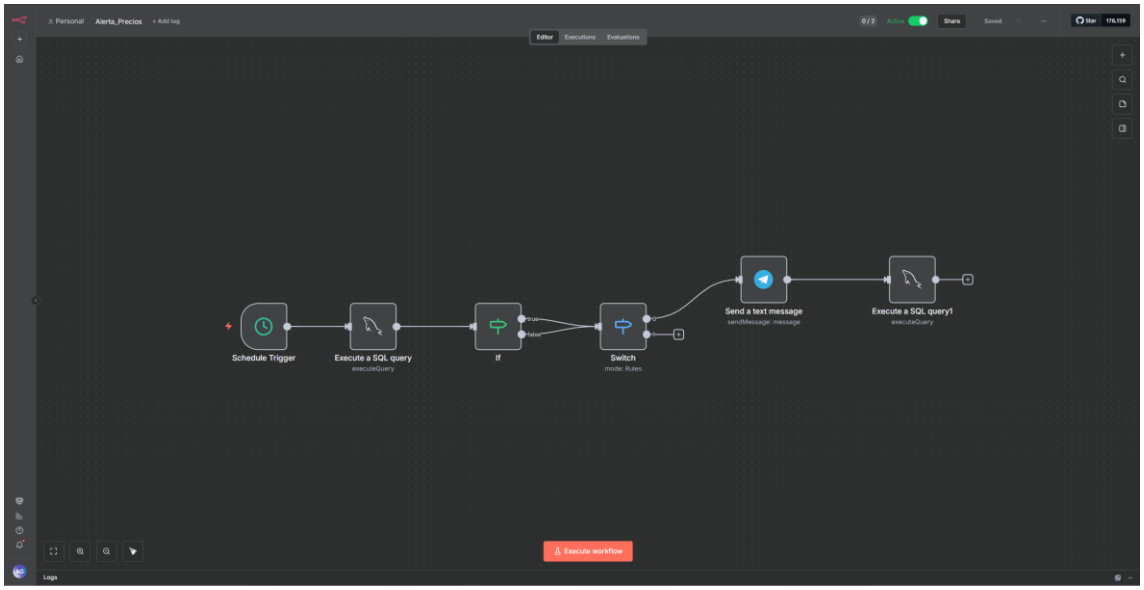
5.1 Automatización con n8n

5.1.1 Normalización de productos con IA (OpenAI)



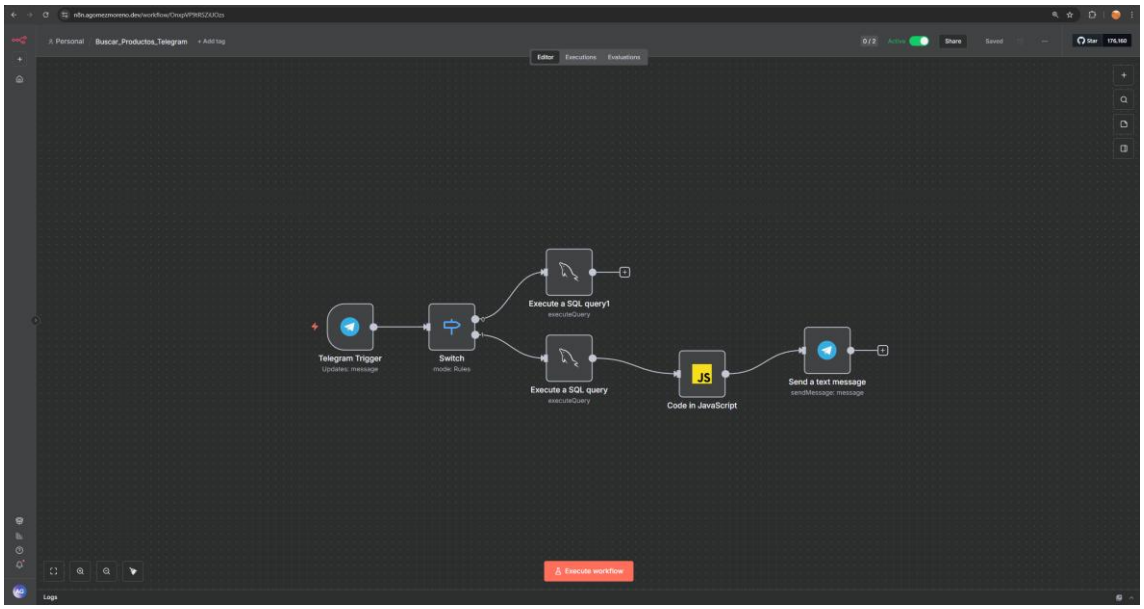
El flujo comienza extrayendo los registros sucios de la tabla PRODUCTOS_JSON mediante un nodo de consulta SQL. Mediante un nodo de OpenAI, se envía el nombre original del supermercado a la API de ChatGPT con un prompt específico para que devuelva el nombre normalizado. Finalmente, un nodo de JavaScript procesa la respuesta para darle el formato correcto y actualiza la tabla PRODUCTO mediante una operación de upsert, evitando así la duplicidad de datos en el sistema.

5.1.2 Flujo de Alertas de Telegram



Este flujo se encarga de monitorizar la base de datos para avisar al usuario cuando un producto baja de precio. El proceso comienza realizando una consulta SQL que compara el precio actual de los productos con el umbral de precio fijado por cada usuario en sus alertas. Si el sistema detecta que la condición se cumple, el flujo activa el nodo de Telegram para enviar un mensaje automático al móvil del usuario. El mensaje incluye el nombre del producto, el nuevo precio detectado y el supermercado donde se encuentra la oferta, permitiendo al usuario ahorrar de forma inmediata sin tener que revisar la web manualmente.

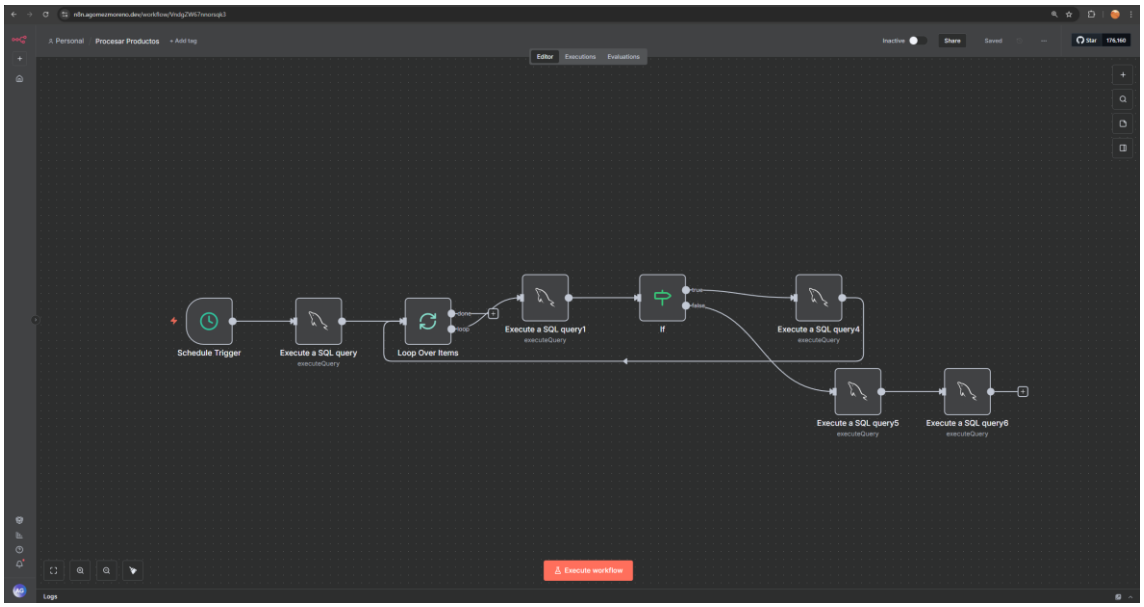
5.1.3 Flujo de Consultas vía Bot de Telegram



Este flujo permite que el usuario consulte precios directamente desde el chat de Telegram. El proceso se activa cuando el nodo Telegram Trigger recibe un mensaje del usuario. Un nodo

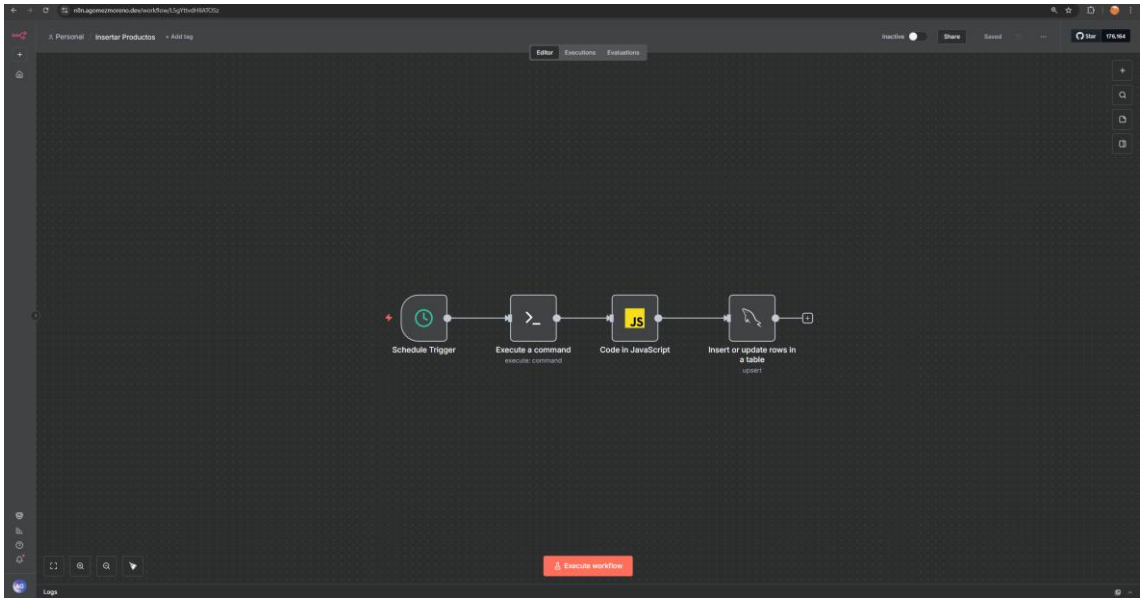
Switch analiza el comando enviado: si es "/start", guarda el identificador del chat para vincularlo a la cuenta del usuario; si es "/buscar", realiza una consulta SQL para encontrar los productos en la base de datos. Finalmente, un nodo de JavaScript organiza la información de los productos encontrados y el bot responde automáticamente con una lista de precios y enlaces de compra.

5.1.4 Flujo de procesamiento



Este flujo es el encargado de organizar la información dentro de la base de datos para evitar que se llenen las tablas con productos repetidos cada vez que se realiza un nuevo scraping. Su función principal es comparar los artículos entrantes con los ya existentes: si el producto ya está registrado, el flujo simplemente actualiza su precio; si es nuevo, lo da de alta de forma automática. Aunque en la fase de desarrollo este flujo se mantiene inactivo para controlar manualmente las pruebas de datos, es una pieza fundamental para garantizar que el sistema sea eficiente y que las alertas de Telegram reciban siempre información real y limpia.

5.1.5 Flujo de inserción de productos



Este flujo es el encargado de recoger los precios reales de los supermercados y guardarlos en el sistema para que puedan ser comparados. El proceso se activa automáticamente de forma periódica mediante un nodo Schedule Trigger, el cual ordena al servidor ejecutar el script de Python que extrae la información actualizada de las webs de Mercadona y Día. Una vez obtenidos los datos, un nodo de JavaScript realiza una limpieza rápida para verificar que no haya errores en los formatos de los precios y, finalmente, un nodo de MySQL guarda toda la información en la tabla de entrada de la base de datos, dejando los productos listos para que el buscador de la web los muestre al usuario.

6. Evaluación y prueba

En este apartado se detallan los resultados de las pruebas realizadas para comprobar la estabilidad del sistema y la precisión de la integración de datos.

6.1 Verificación de funciones

- **Sincronización de imágenes**: Se ha comprobado que el sistema muestra correctamente las imágenes correspondientes a cada producto y supermercado, facilitando la identificación visual para el usuario.

- **Alertas de Telegram**: Las pruebas de conectividad entre la base de datos, n8n y el bot de Telegram han sido satisfactorias, recibiendo el aviso en el móvil al detectar cambios de precio.

- **Gestión de usuarios**: Se ha verificado que el flujo de registro e inicio de sesión con Firebase funciona correctamente y protege las funciones de alertas.

6.2 Limitaciones detectadas (Errores de IA)

Durante las pruebas de normalización con la IA en n8n, se ha identificado un comportamiento técnico a mejorar:

- **Duplicidad de datos por ambigüedad**: En productos con nombres muy similares, la IA tiende a asignar la URL y el precio del primer resultado encontrado a los productos equivalentes.

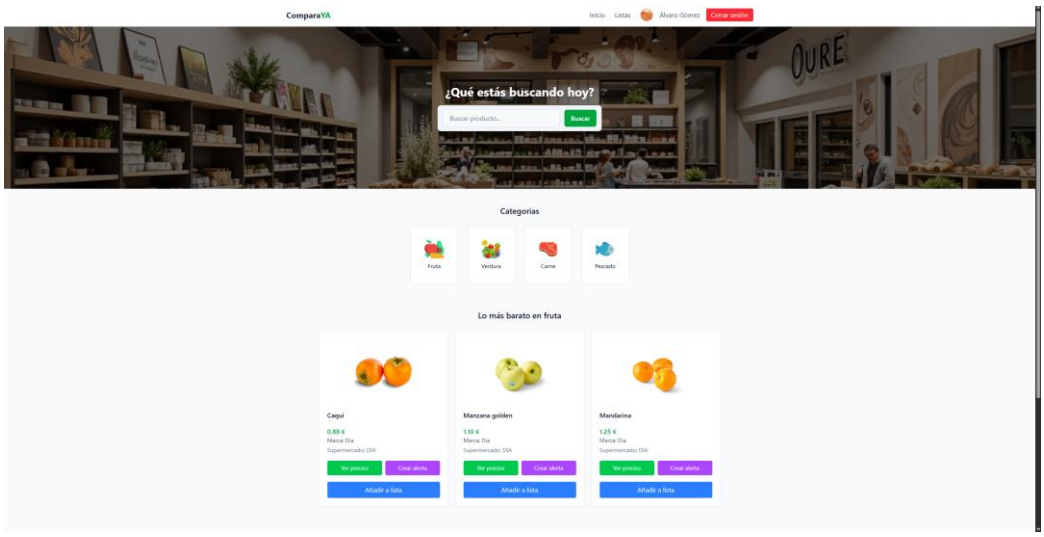
- **Origen del error**: Este fallo proviene de la dificultad de la IA para diferenciar registros casi idénticos durante el proceso de normalización en el flujo de n8n. No obstante, al mantener las imágenes correctas, el prototipo permite validar la estructura de la comparativa, quedando la corrección de los enlaces y precios duplicados como una mejora para futuras versiones.

7. Capturas de pantalla

En este apartado se presentan las capturas de pantalla definitivas de ComparaYA, mostrando el resultado visual final y el flujo que sigue el usuario al utilizar la plataforma.

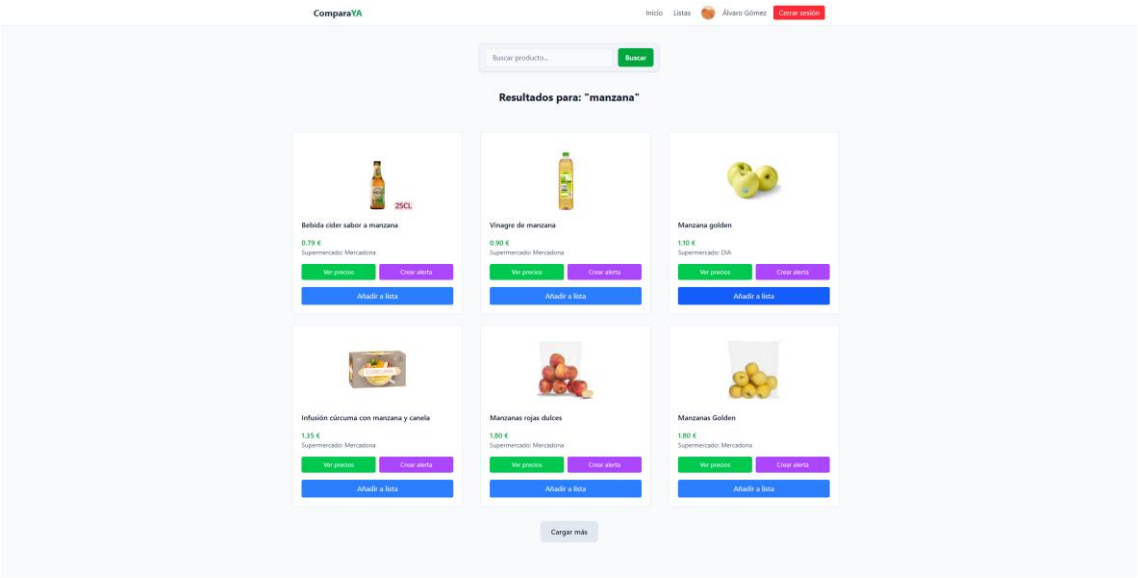
7.1 Inicio

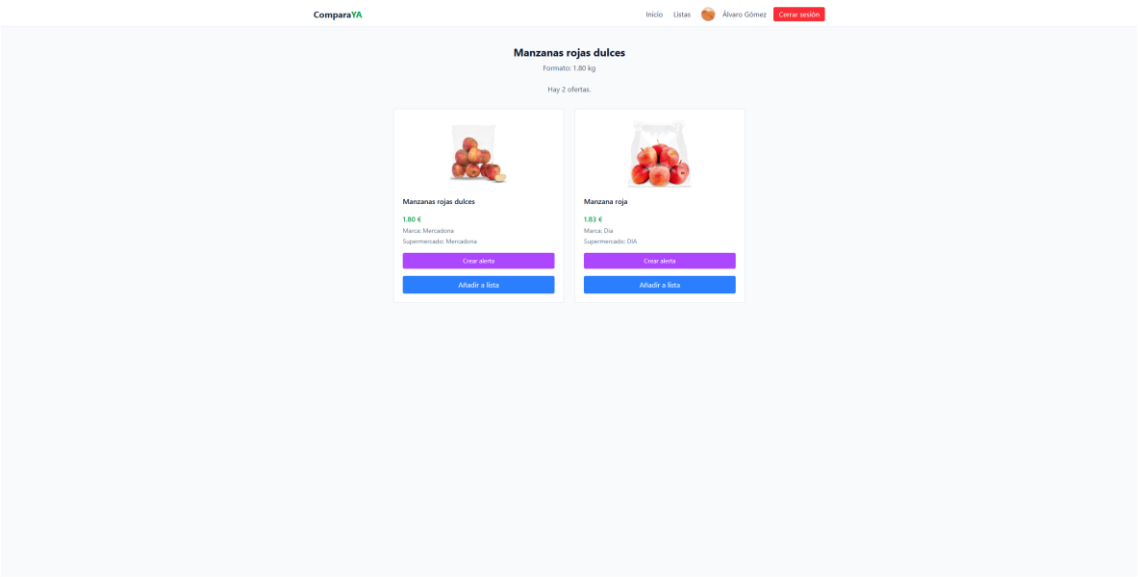
La pantalla de inicio permite al usuario comenzar la búsqueda de productos de forma rápida. En la parte superior se encuentra el menú con acceso al inicio, la lista de compra y el perfil en caso de haber iniciado sesión o el botón Login en caso de no haber iniciado sesión. El elemento principal es un buscador donde el usuario escribe el producto que desea comparar. Debajo se muestran categorías para acceder más fácilmente a grupos de productos como fruta, verdura, carne o pescado. También se presentan algunas ofertas destacadas del día en forma de tarjetas con nombre, marca, supermercado, precio y un botón para añadir a la lista y crear alerta. Al final, se incluye la sección de Cómo funciona que explica brevemente los pasos básicos: buscar productos, crear la lista y ver donde resulta más barata la compra.



7.2 Resultados

La búsqueda se divide en dos pantallas. La primera muestra los productos normalizados que coinciden con el término introducido por el usuario, destacando cuál es el más barato en ese momento y permitiendo acceder a la comparación detallada a través del botón “Ver precios”. La segunda pantalla muestra la comparación directa de precios del mismo producto en diferentes supermercados, indicando nombre, precio, marca y supermercado además de permitir añadir el producto a la lista de compra y crear alertas.

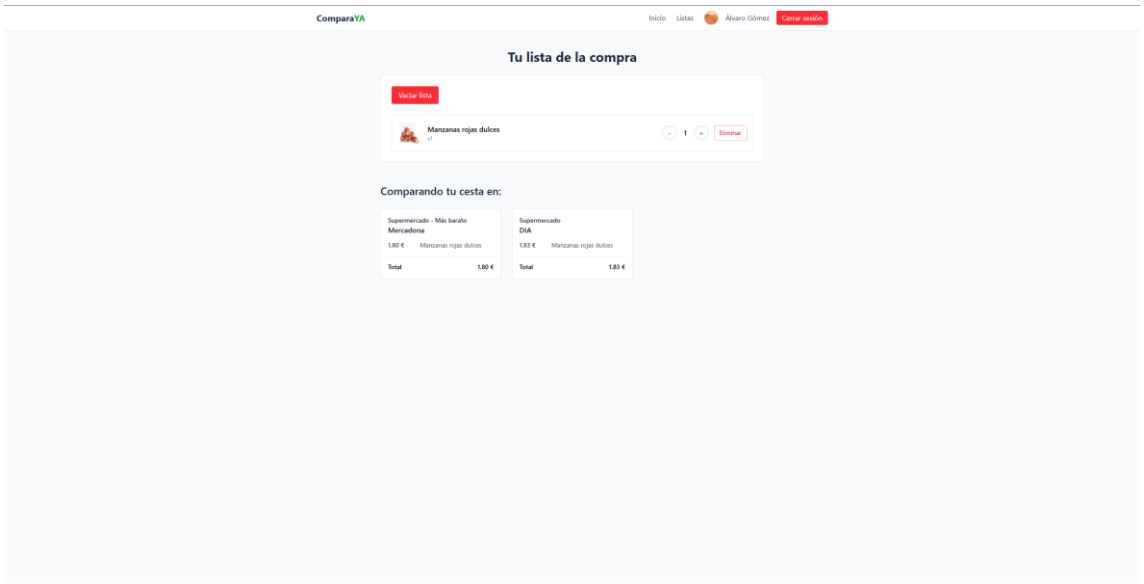




7.3 Lista compra

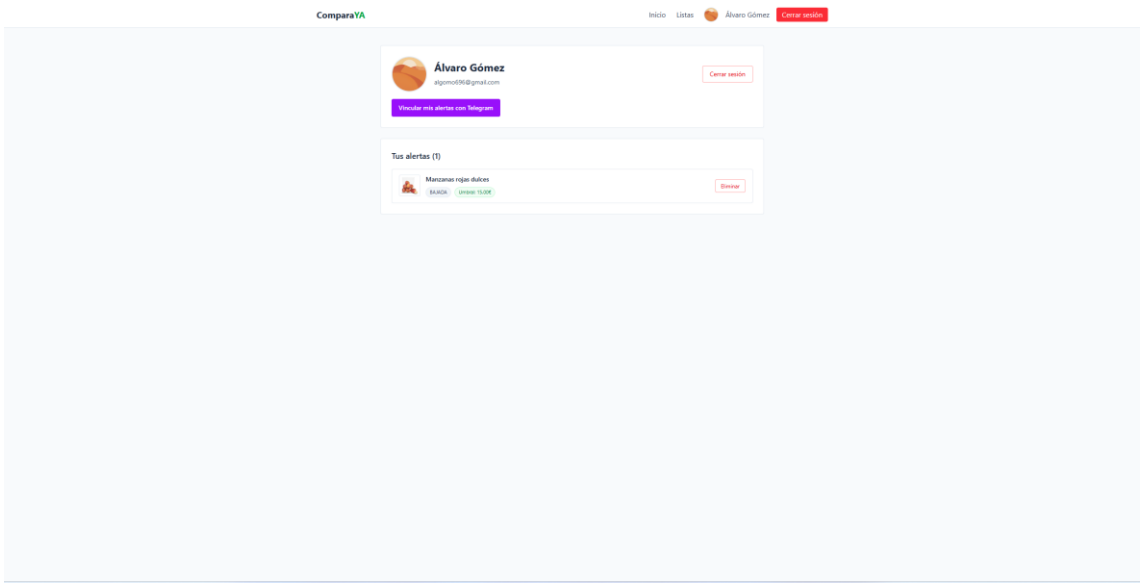
En esta pantalla se muestran los productos añadidos por el usuario, indicando nombre, imagen y cantidad, con opción para aumentar, disminuir o eliminar cada artículo. Debajo, aparece la sección “Comparando cesta en:”, donde la aplicación calcula automáticamente el precio total de la misma lista en distintos supermercados. Se muestran los totales y se destaca cuál es el

más barato, ayudando al usuario a decidir dónde comprar para ahorrar, también existe un botón donde puedes añadir más cantidad o restar al mismo producto y un botón donde puedes vaciar la lista, además de otro para eliminar un producto de la lista.



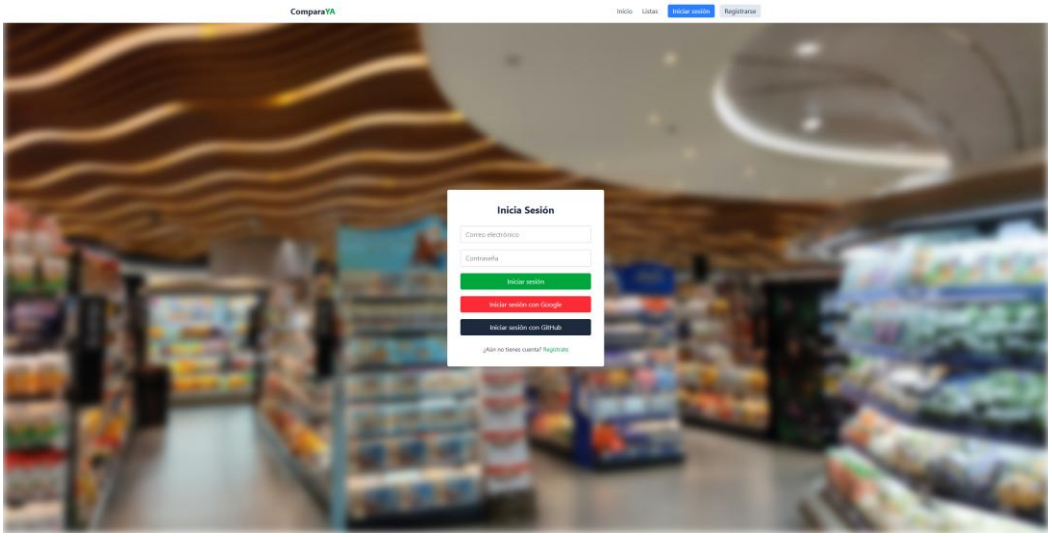
7.4 Perfil

Esta pantalla solo está disponible para usuarios registrados. Aquí se muestran los datos básicos de la cuenta (nombre y correo) y se permite gestionar contenido personal. En esta pantalla se pueden vincular las alertas a Telegram para recibir los avisos en el móvil, ver las alertas que están activas actualmente y eliminarlas si ya no se necesitan. Finalmente, se incluye un botón de cerrar sesión para salir de la cuenta de forma segura.



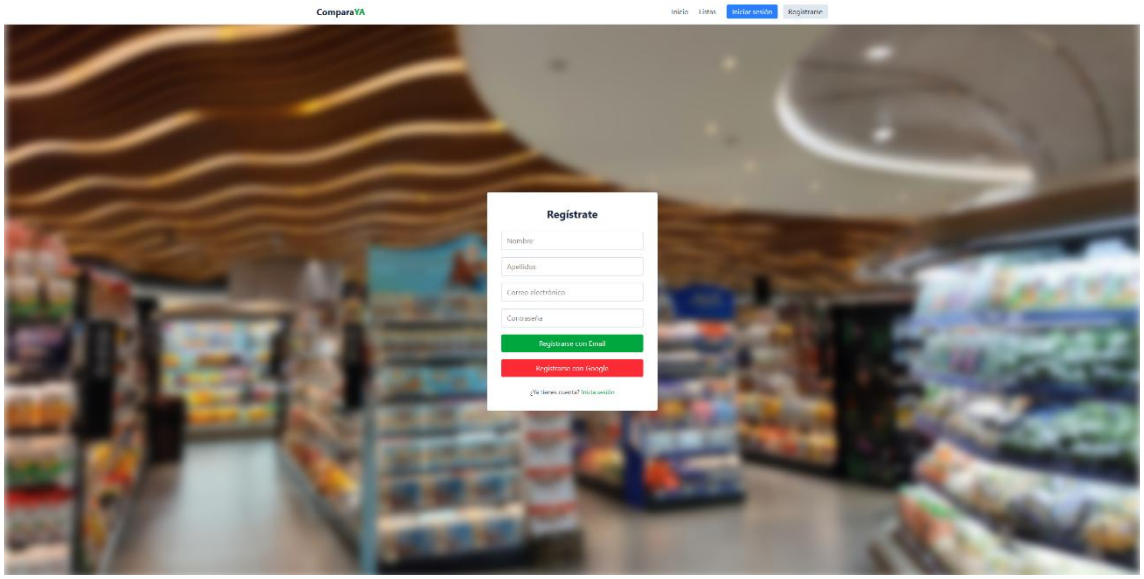
7.5 Inicio de sesión

En esta pantalla el usuario puede acceder a su cuenta introduciendo su correo y contraseña. Además, se ofrece la opción de crear cuenta si aún no está registrado. Además, se incluyen dos botones para iniciar sesión mediante Google o GitHub, facilitando el acceso sin necesidad de recordar credenciales. Una vez iniciado, el usuario podrá gestionar su perfil y configurar alertas personalizadas



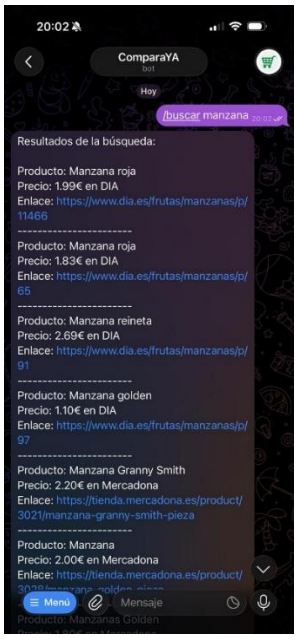
7.6 Registro

Esta pantalla permite crear cuenta rellenando los datos básicos del usuario nombre, apellidos, correo y contraseña. Una vez completado el formulario, el usuario puede pulsar Crear cuenta para registrarse. También se ofrece acceso directo para iniciar sesión si ya tiene cuenta, así como como la opción de registrarse usando Google o con una dirección de correo normal para mayor comodidad.



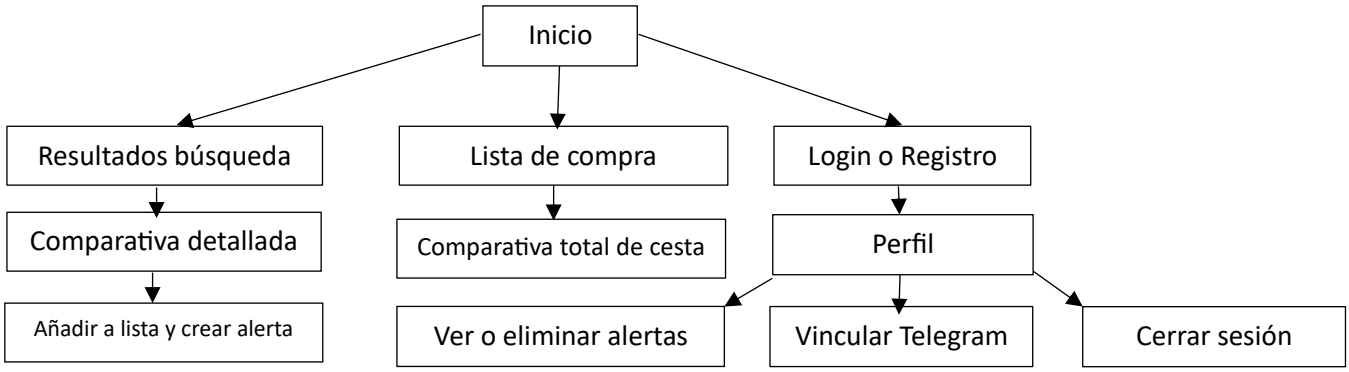
7.7 Interfaz del Bot de Telegram

Consultas en tiempo real: En esta captura muestro el resultado del comando /buscar manzana. Se observa cómo el bot devuelve un listado estructurado que incluye el nombre exacto del producto, el precio actual, el supermercado de origen y un enlace directo a la tienda para facilitar la compra inmediata.



Alertas automáticas: A continuación, se puede apreciar el funcionamiento del sistema de alertas. El bot envía una notificación automática cuando detecta que un producto guardado por el usuario en este caso, "Manzanas rojas dulces" ha bajado de precio, indicando el nuevo importe y comparándolo con el umbral de precio que el usuario configuró previamente.

8. Mapa de navegación



9. Conclusiones y Mejoras posibles

El desarrollo de ComparaYA me ha permitido validar que es posible crear una herramienta funcional que centralice los precios de los supermercados para facilitar el ahorro. Ha sido un desafío técnico integrar tecnologías tan diferentes como React, Node.js y Firebase, MySQL con la automatización de n8n, pero el resultado es un sistema sólido y capaz de procesar datos reales.

Lo más difícil ha sido la normalización de productos mediante IA. Aunque he detectado que el sistema todavía presenta margen de mejora al diferenciar productos con nombres casi iguales que a veces provoca duplicidad en precios o URLs. El uso de Telegram como canal de comunicación ha resultado ser un acierto por su inmediatez, y Firebase ha aportado la seguridad necesaria en la gestión de los usuarios.

9.1 Mejoras posibles y aportaciones

Como el proyecto se encuentra en fase de prototipo, he identificado varios puntos para seguir evolucionando la aplicación:

- **Optimización de la IA**: Ajustar la lógica en n8n para que la normalización sea más precisa y evitar errores en la asignación de enlaces y precios en productos muy similares.

- **Ciclo de vida de las alertas**: Programar la eliminación automática de la alerta una vez notificada al usuario por Telegram, evitando así avisos redundantes y mejorando la experiencia de uso.

- **Automatización del scraping**: Configurar tareas programadas para que la actualización de precios sea totalmente autónoma y diaria, sin necesidad de intervención manual.

- **Aportación profesional**: La herramienta tiene un potencial claro para perfiles comerciales y del sector alimentario (como el caso de mi tío), ya que permite monitorizar la competencia de forma rápida para la toma de decisiones.

- **Integración de lectura de códigos de barras**: Una de las mejoras con más potencial es transformar la web en una aplicación móvil capaz de escanear el código de barras de un producto directamente en el estante del supermercado. Esto permitiría al usuario saber, en tiempo real, si ese mismo artículo está más barato en otra tienda sin necesidad de buscarlo manualmente, mejorando drásticamente la utilidad de ComparaYA durante la compra física.

- **Gestión de múltiples listas**: Actualmente, el sistema permite tener una lista de la compra activa que se mantiene guardada en la cuenta del usuario incluso si cierra sesión. Una mejora útil sería permitir que el usuario pueda crear y guardar varias listas diferentes con nombres personalizados, para que pueda organizar sus compras por categorías o eventos sin tener que borrar la lista actual.